Name: **Yogita Dinesh Patil**
Class: **D20A**
Roll No: **57**

Lab - 1: **Experiment No: 1**

AIM: Cryptography in Blockchain, Merkle root Tree Hash

Lab Objectives: To explore Blockchain concepts.

***Lab Outcomes (LO)*:** Creating Cryptographic Hash using Merkle Tree (LO1)

Task to be performed :

1. Make a copy of this **Google Colab Notebook**
2. Try to solve the errors in each of the 4 Programs
3. In the 4th Program - Constructing a Merkle Tree Root Hash, modify the code as follows:
    - Update the transactions list with valid entries.

      eg : transactions = ['A -> B : $10', 'B -> C : $5,'C -> A : $2']

      Sample Transactions to be considered

      T1 : Alice → Bob : $200; T2 : Bob → Dave : $500; T3 : Dave → Eve : $100

      T4 : Eve → Alice : $300; T5 : Roo → Bob : $50
    - Hash the transactions before combining them in the for loop
    - Print all the intermediate hash during the construction of the Merkle Tree Root Hash

Tools & Libraries used :

- Python Libraries: **hashlib**

Instructions :

1. **Cryptographic Hash Functions in Blockchain**

   A cryptographic hash function takes an input of any size and converts it into a unique, fixed-size string of characters. In blockchain, it secures data integrity because even a tiny change to the transaction data will completely change the resulting hash. This makes the ledger tamper-proof since any alterations are instantly detected.

2. **What is a Merkle Tree?**

   A Merkle Tree is a hierarchical data structure in cryptography where individual data blocks are hashed and organized into a tree shape. It continuously pairs and hashes data points until they culminate in a single hash at the top called the Merkle Root. This allows networks to verify large sets of data quickly and securely.

3. **What is a Cryptographic Puzzle and explain the Golden Nonce**

   A cryptographic puzzle is a computational challenge that blockchain miners must solve to validate transactions and add a new block. The Golden Nonce is the exact arbitrary number that, when combined with the block data and hashed, successfully produces a hash value that meets the network's strict difficulty target.

4. **How does a Merkle Tree work?**

   It starts by hashing individual transactions at the bottom leaf level, then pairs adjacent hashes together to create a new combined hash. This process repeats recursively up the structure, layer

by layer, until only one final hash—the Merkle Root—remains to represent the entire dataset.

5. **Benefits of Merkle Tree**

   High Efficiency: Verifies specific transactions without downloading the entire blockchain ledger.

   Low Storage: Minimizes the memory and disk space needed for data verification.

   Tamper Proof: Ensures any data alteration instantly breaks the entire hash chain.

6. **Use Cases of Merkle Tree**

   Simplified Payment Verification (SPV): Enables light crypto wallets to verify transactions quickly.

   Data Synchronization: Helps peer-to-peer networks verify and repair inconsistent blocks of data.

   Hyperledger Fabric: Minimizes computational overhead when auditing system state histories.

Outcome :

1. Understood the concept of hashing, Cryptographic Puzzle to find nonce, Merkle Tree and its relevance.
2. Implemented programs
   - To find the Golden Nonce while solving the Cryptographic Puzzle
   - To construct a Merkle Tree for the given transactions
3. Prepare a document with Aim, Tasks performed, Program, Output and Conclusion.
4. Submit the hardcopy **by the 1st week of Sem**

   (As per the instructions, submit a hard copy of the same).

Aim: Cryptography in Blockchain, Merkle root Tree Hash

Tasks Performed:

1. **Hash Generation using SHA-256:**
   - Developed a Python program to compute a SHA-256 hash for any given input string using the `hashlib` library.
2. **Target Hash Generation with Nonce:**
   - Created a program to generate a hash code by concatenating a user input string and a nonce value to simulate the mining process.
3. **Proof-of-Work Puzzle Solving:**
   - Implemented a program to find the nonce that, when combined with a given input string, produces a hash starting with a specified number of leading zeros.
4. **Merkle Tree Construction:**
   - Built a Merkle Tree from a list of transactions by recursively hashing pairs of transaction hashes, doubling up last nodes if needed, and generated the Merkle Root hash for blockchain transaction integrity.

Program and Output

Python program that uses the hashlib library to create the hash of a given string

```
import hashlib

def create_hash(string):
    # Create a hash object using SHA-256 algorithm
    hash_object = hashlib.sha256()
    # Convert the string to bytes and update the hash object
    hash_object.update(string.encode('utf-8'))
    # Get the hexadecimal representation of the hash
    hash_string = hash_object.hexdigest()
    # Return the hash string
    return hash_string

# Example usage
input_string = input("Enter a string: ")
hash_result = create_hash(input_string)
print("Hash:", hash_result)
```

```
Enter a string: yogita dinesh patil
Hash: 1532cf3a8c34f89aa2a1f0052784ccf747e36a902aa68fab2526d6b2a0d9b7fd
```

Python code for Solving Puzzle for leading zeros with expected nonce and given string

```
import hashlib

def find_nonce(input_string, num_zeros):
    nonce = 0
    hash_prefix = '0' * num_zeros

    while True:
        # Concatenate the string and nonce
        hash_string = input_string + str(nonce)
        # Calculate the hash using SHA-256
        hash_object = hashlib.sha256(hash_string.encode('utf-8'))
        hash_code = hash_object.hexdigest()

        # Check if the hash code has the required number of leading zeros
        if hash_code.startswith(hash_prefix):
            print("Hash:",hash_code )
            return nonce

        nonce += 1

# Get user input
input_string = "Yogita"
num_zeros = 6

# Find the expected nonce
expected_nonce = find_nonce(input_string, num_zeros)

# Print the expected nonce
print("Input String:", input_string)
print("Leading Zeros:", num_zeros)
print("Expected Nonce:", nonce)
```

```
Hash: 000000895f38c6f073c4676ac7ef80d5088293b3d5653e0f5962d89ce58b0f78
Input String: Yogita
Leading Zeros: 6
Expected Nonce: 20042459
```

Program to generate required target hash with input string and nonce

```python
import hashlib

# Get user input
input_string = input("Enter a string: ")
nonce = input("Enter the nonce: ")

# Concatenate the string and nonce
hash_string = input_string + nonce

# Calculate the hash using SHA-256
hash_object = hashlib.sha256(hash_string.encode('utf-8'))
hash_code = hash_object.hexdigest()

# Print the hash code
print("Hash Code:", hash_code)
```

```
Enter a string: Patil
Enter the nonce: 86689

Hash: 00000aa726ee1b6f05cc121bd21dfce825a3c9fdfeba76370725e3cbe404ce58
```

Generating Merkle Tree for given set of Transactions

```python
import hashlib

def build_merkle_tree(transactions):
    if len(transactions) == 0:
        return None

    if len(transactions) == 1:
        return transactions[0]

    # Recursive construction of the Merkle Tree
    while len(transactions) > 1:
        if len(transactions) % 2 != 0:
            transactions.append(transactions[-1])

        new_transactions = []
        for i in range(0, len(transactions), 2):
            combined = transactions[i] + transactions[i+1]
            hash_combined = hashlib.sha256(combined.encode('utf-8')).hexdigest()
            new_transactions.append(hash_combined)

        transactions = new_transactions

    return transactions[0]

# Example usage
transactions = ["Transaction 1", "Transaction 2", "Transaction 3", "Transaction 4", "Transaction 5"]

merkle_root = build_merkle_tree(transactions)
print("Merkle Root:", merkle_root)
```

```
Merkle Root: a4a18941de1162b17a46c4f8c87d8a0850b46fad17ac881340061d9233785077
```

Conclusion:

This study demonstrates fundamental cryptographic concepts and their applications within blockchain technology:

- **SHA-256 Hashing** provides a secure and unique fingerprint for any input, ensuring data integrity.
- **Nonce and Target Hashing** allow the simulation of proof-of-work consensus, emphasizing the difficulty of mining blocks by meeting hash conditions (leading zeros).
- **Proof-of-Work Puzzle Solving** shows how mining requires computational effort to find a nonce making the hash satisfy network difficulty.
- **Merkle Tree Construction** efficiently summarizes and validates large numbers of transactions with a single Merkle root hash, which is essential in blockchain for fast verification and ensuring the integrity of data blocks.

Together, these implementations capture the essence of how cryptography underpins blockchain's security, immutability, and efficiency, especially through hash functions and structures like Merkle Trees that enable trustless and decentralized transaction management.